

Étape 2 – Comprendre les données et modéliser la base

Projet SportDataPulse – Analyse des performances des joueurs et des équipes Ligue 1 – Saison 2025

Ce livrable présente la compréhension des sources de données, les constats de qualité, le modèle logique retenu et le dictionnaire de données cible. Il prépare la création de la base PostgreSQL, l'import automatisé des CSV et la réalisation des requêtes SQL demandées par le directeur sportif.

1. Périmètre de l'étape 2

Cette étape consiste à analyser le fichier Excel source, identifier les entités exploitables, définir le grain des données et proposer une modélisation relationnelle cohérente pour les analyses de performance des joueurs de Ligue 1.

Les données disponibles sont agrégées au niveau du joueur sur la saison. Elles ne correspondent pas à des événements de match détaillés : le modèle permet donc des analyses globales par joueur, poste et équipe, mais pas une analyse action par action.

- Table centrale : joueur
- Tables de référence : equipe, position
- Tables statistiques : temps_de_jeu, performance, attendu, progression
- Source complémentaire : salaire_source, avec rapprochement vers joueur lorsque la correspondance est fiable

2. Inventaire des fichiers et tables analysés

Les volumes ci-dessous correspondent aux CSV préparés pour l'import. Les colonnes redondantes présentes dans certains fichiers sources sont ignorées lorsqu'elles peuvent être retrouvées par jointure ou recalculées en base.

Table / fichier CSV	Nb lignes utiles	Nb colonnes utiles importées	Principaux champs importés	Identifiant repéré
joueur	551	6	id_joueur, code_position, id_equipe, nom_joueur, nation, annee_naissance	id_joueur
temps_de_jeu	551	5	id_joueur, matchs_joues, matchs_titulaires, minutes_jouees, matchs_90	id_joueur
performance	551	8	id_joueur, buts, passes_decisives, buts_hors_penalties, penalties_marques, penalties_tentes, cartons_jaunes, cartons_rouges	id_joueur
attendu	551	4	id_joueur, xg, npxg, xag	id_joueur
progression	551	4	id_joueur, progression_conduite, progression_passe, progression_cOURSE	id_joueur
position	16	2	code_position, libelle_position	code_position
equipe	18	2	id_equipe, nom_equipe	id_equipe
salaire_source	105	5 importées + id_joueur réconcilié	id_joueur_source, nom_joueur, age, salaire_annuel, note_source, id_joueur	id_joueur_source

3. Constats de qualité des données

- Le fichier joueur contient 551 lignes et constitue le référentiel principal des joueurs utilisés dans les analyses.
- Les tables statistiques principales contiennent chacune 551 lignes exploitables et peuvent être reliées à joueur via id_joueur.
- Les données sont agrégées à l'échelle de la saison : il n'y a pas de détail par match, par date ou par événement de jeu.

- Certaines colonnes présentes dans les CSV sont redondantes. Par exemple, l'équipe peut être retrouvée via la relation joueur -> equipe.
- Certains indicateurs sont calculables, notamment buts_plus_passes et npxg_xag. Ils sont gérés comme colonnes générées ou recalculés dans les requêtes.
- Les nombres décimaux issus des CSV utilisent parfois la virgule française. L'import SQL prévoit une conversion vers le format NUMERIC de PostgreSQL.
- La table salaire_source possède ses propres identifiants. Le rapprochement avec joueur est réalisé par nom normalisé, éventuellement complété par l'âge et des contrôles manuels.

4. Modèle logique recommandé

Le modèle logique repose sur une table centrale JOUEUR. Les tables EQUIPE et POSITION jouent le rôle de dimensions de référence. Les tables TEMPS_DE_JEU, PERFORMANCE, ATTENDU et PROGRESSION stockent les indicateurs statistiques liés aux joueurs.

Chaque table statistique dispose d'un identifiant technique propre et conserve id_joueur comme clé étrangère. Cette structure facilite les imports, sécurise les jointures et évite de dépendre d'un champ statistique comme clé primaire.

La table SALAIRE_SOURCE reste séparée afin de conserver la traçabilité de la source salariale. Le champ id_joueur y est nullable : il est renseigné uniquement lorsque la correspondance avec la table joueur est suffisamment fiable.

Relation	Cardinalité
position -> joueur	1-N
equipe -> joueur	1-N
joueur -> temps_de_jeu	1-N
joueur -> performance	1-N
joueur -> attendu	1-N
joueur -> progression	1-N
joueur -> salaire_source	0-N après réconciliation

Figure 1 – Schéma relationnel simplifié de la base SportDataPulse.

5. Relations retenues

Table source	Table cible	Cardinalité	Justification
position	joueur	1-N	Un code de position peut être associé à plusieurs joueurs.
equipe	joueur	1-N	Une équipe regroupe plusieurs joueurs.
joueur	temps_de_jeu	1-N	Les lignes de temps de jeu sont rattachées à un joueur via id_joueur.
joueur	performance	1-N	Les lignes de performance sont rattachées à un joueur via id_joueur.
joueur	attendu	1-N	Les statistiques attendues sont rattachées à un joueur via id_joueur.
joueur	progression	1-N	Les indicateurs de progression sont rattachés à un joueur via id_joueur.
joueur	salaire_source	0-N	Le rattachement est optionnel et dépend de la réconciliation entre la source salaire et la table joueur.

6. Dictionnaire de données cible

Le dictionnaire ci-dessous décrit la structure cible de la base de données.

POSITION

Champ	Type SQL conseillé	Contraintes	Description
code_position	VARCHAR(10)	PK	Code de position principal ou combiné.
libelle_position	VARCHAR(50)	NOT NULL	Libellé de la position.

EQUIPE

Champ	Type SQL conseillé	Contraintes	Description
id_equipe	INTEGER	PK	Identifiant unique du club.
nom_equipe	VARCHAR(100)	NOT NULL, UNIQUE	Nom du club.

JOUEUR

Champ	Type SQL conseillé	Contraintes	Description
id_joueur	INTEGER	PK	Identifiant joueur issu des données sources.
nom_joueur	VARCHAR(100)	NOT NULL	Nom complet du joueur.
nation	VARCHAR(50)	NULL	Nationalité du joueur.
annee_naissance	INTEGER	NULL	Année de naissance.
code_position	VARCHAR(10)	FK -> POSITION(code_position)	Poste principal ou code de position.
id_equipe	INTEGER	FK -> EQUIPE(id_equipe)	Club d'appartenance.

TEMPS_DE_JEU

Champ	Type SQL conseillé	Contraintes	Description
id_temps_de_jeu	SERIAL / IDENTITY	PK	Identifiant technique de la ligne de temps de jeu.
id_joueur	INTEGER	FK -> JOUEUR(id_joueur), NOT NULL	Identifiant du joueur concerné.
matches_joues	INTEGER	CHECK >= 0	Nombre de matchs joués.
matches_titulaires	INTEGER	CHECK >= 0	Nombre de matchs débutés.
minutes_jouees	INTEGER	CHECK >= 0	Temps total en minutes.
matches_90	NUMERIC(6,2)	CHECK >= 0	Équivalent nombre de matchs complets.

PERFORMANCE

Champ	Type SQL conseillé	Contraintes	Description
id_performance	SERIAL / IDENTITY	PK	Identifiant technique de la ligne de performance.
id_joueur	INTEGER	FK -> JOUEUR(id_joueur), NOT NULL	Identifiant du joueur concerné.
buts	INTEGER	CHECK >= 0	Nombre total de buts.
passes_decisives	INTEGER	CHECK >= 0	Nombre de passes décisives.
buts_plus_passes	GENERATED	Calculé	Indicateur calculé : buts + passes_decisives.
buts_hors_penalties	INTEGER	CHECK >= 0	Buts hors penalties.
penalties_marques	INTEGER	CHECK >= 0	Penalties marqués.
penalties_tentes	INTEGER	CHECK >= 0	Penalties tentés.
cartons_jaunes	INTEGER	CHECK >= 0	Cartons jaunes.
cartons_rouges	INTEGER	CHECK >= 0	Cartons rouges.

ATTENDU

Champ	Type SQL conseillé	Contraintes	Description
id_attendu	SERIAL / IDENTITY	PK	Identifiant technique de la ligne de statistiques attendues.
id_joueur	INTEGER	FK -> JOUEUR(id_joueur), NOT NULL	Identifiant du joueur concerné.
xg	NUMERIC(6,2)	CHECK >= 0	Expected Goals.
npvg	NUMERIC(6,2)	CHECK >= 0	Expected Goals hors penalties.
xag	NUMERIC(6,2)	CHECK >= 0	Expected Assists.
npvg_xag	GENERATED	Calculé	Indicateur calculé : npvg + xag.

PROGRESSION

Champ	Type SQL conseillé	Contraintes	Description
id_progression	SERIAL / IDENTITY	PK	Identifiant technique de la ligne de progression.
id_joueur	INTEGER	FK -> JOUEUR(id_joueur), NOT NULL	Identifiant du joueur concerné.
progression_conduite	INTEGER	CHECK >= 0	Courses progressives.
progression_passe	INTEGER	CHECK >= 0	Passes progressives.
progression_course	INTEGER	CHECK >= 0	Passes reçues progressives.

SALAIRE_SOURCE

Champ	Type SQL conseillé	Contraintes	Description
id_joueur_source	INTEGER	PK	Identifiant propre à la source salaire.
nom_joueur	VARCHAR(100)	NOT NULL	Nom du joueur dans la source salaire.
age	INTEGER	NULL, CHECK >= 0	Âge du joueur dans la source salaire.
salaire_annuel	NUMERIC(12,2)	CHECK >= 0	Montant du salaire annuel.
note_source	TEXT	NULL	Commentaire sur la qualité de la donnée.
id_joueur	INTEGER	NULL, FK -> JOUEUR(id_joueur)	Identifiant joueur renseigné après réconciliation.

7. Règles de modélisation et d'import

1. Les CSV sont extraits depuis le fichier Excel source et importés dans PostgreSQL.
2. Les lignes vides et colonnes vides parasites peuvent être supprimées lorsqu'elles empêchent l'import technique.
3. Les champs sont renommés selon une convention SQL homogène : minuscules, sans espaces ni accents.
4. Les colonnes redondantes des CSV sources ne sont pas conservées dans les tables cibles lorsqu'elles peuvent être retrouvées par jointure.
5. Les indicateurs calculables, notamment buts_plus_passes et npxg_xag, sont définis en colonnes générées ou recalculés dans les requêtes.
6. Les décimaux au format français sont convertis lors de l'import avec REPLACE(valeur, ',', '.') avant conversion en NUMERIC.
7. Les fichiers CSV utilisent le point-virgule comme séparateur.
8. La table salaire_source est rapprochée de joueur via le nom normalisé, éventuellement complété par l'âge. Les lignes non réconciliées restent conservées pour traçabilité.
9. Des contrôles de cohérence sont réalisés après chargement : volumes par table, validité des jointures, valeurs nulles critiques, salaires non rattachés et cohérence des indicateurs calculés.

8. Limites du modèle

- Les données sont agrégées à l'échelle de la saison : le modèle ne permet pas d'analyser l'évolution match par match.
- Il n'existe pas de dimension temporelle détaillée permettant d'étudier la forme, les blessures ou la progression au fil de la saison.
- Les données ne contiennent pas d'événements de jeu : tirs, passes, dribbles, séquences ou contexte tactique.
- La donnée salariale est issue d'une source complémentaire et peut contenir des estimations ou des joueurs absents de la table principale.
- Le rapprochement par nom peut rester imparfait en cas d'homonymie, d'accents ou de variations orthographiques.
- Une évolution future du modèle pourrait intégrer des tables supplémentaires comme match, evenement, tir ou passe afin de permettre une analyse plus fine et plus dynamique.

9. Conclusion de l'étape 2

La modélisation proposée permet de structurer les données de Ligue 1 autour d'une table centrale JOUEUR, enrichie par des tables statistiques spécialisées et des dimensions de référence. Elle limite les redondances, facilite les jointures et prépare les analyses SQL attendues.

Le principal point de vigilance concerne la table salaire_source, dont les identifiants ne correspondent pas directement à ceux de la table joueur. Son intégration repose donc sur un rapprochement contrôlé afin de préserver la fiabilité des analyses de recrutement.

Ce modèle constitue la base de la phase suivante : création des tables, import automatisé des CSV, contrôles de cohérence, puis réalisation des requêtes SQL et mise à jour des livrables de présentation.